I/O Techniques

Lecture Notes — Week 8

PCC CS-401: Computer Organization and Architecture

August 12, 2026

8.1 Introduction: The I/O Speed-Mismatch Problem

Weeks 6 and 7 built two different implementations of the same control unit — hardwired (Week 6) and microprogrammed (Week 7) — and both answered the same question: how does the CPU generate its own control-step sequence, cycle by cycle? Both designs quietly assumed the CPU is the only thing that matters: registers, an ALU, and an internal bus. Neither design said anything about what happens when the CPU needs data from, or must send data to, something *outside* the chip — a keyboard, a disk, a printer.

That gap matters because of a raw speed mismatch. A CPU executes roughly one instruction per nanosecond — about 10^9 instructions per second. A keyboard produces roughly one keystroke every 200 milliseconds — about 5 keystrokes per second. The CPU is roughly *200 million times* faster than the typist; the two operate on completely different time scales. If the CPU is responsible for reading every keystroke the instant it arrives, and it does nothing else in between, it wastes an enormous number of cycles waiting. I/O design is therefore really the question of *who does the waiting*, and *what the waiting costs*. This chapter is organized around four different answers to that one underlying question — *who watches the device, and how much of the CPU does the watching cost?* — moving from a purely software answer to a purely hardware one:

1. **Program-controlled I/O (polling)** (Section 8.2) — the CPU repeatedly asks a device "are you ready?" in a loop.
2. **Interrupt-driven I/O** (Section 8.3) — the device asks the CPU instead, only when it actually has something to report.
3. **Synchronous vs. asynchronous buses** (Section 8.4) — one level below the programmer's view: how does the wire-level handshake between a bus master and a bus slave actually work?
4. **DMA and I/O processors** (Section 8.5) — what if the CPU is removed from the data-transfer path entirely?

Two worked examples thread through the whole chapter and tie the four answers together. A keyboard-input/display-output pair, using the status flags `KIN` (keyboard input ready) and `DOUT` (display output ready), runs through Sections 8.2 and 8.3 to compare the programmer's-view techniques. A 4096-byte block transfer runs through Sections 8.4 and 8.5, first illustrating synchronous/asynchronous bus timing and then motivating DMA, where a three-device bus-arbitration example also appears. Anchor reading for this chapter is Hamacher, *Computer Organization and Embedded Systems*, 6th ed., Ch. 3 (the programmer's view of I/O, p.95–119) and Ch. 7 §§7.1–7.4 (the hardware view: bus structure and operation, p.227–242) [1].

8.2 The Programmer's View: Memory-Mapped I/O and Program-Controlled Polling

8.2.1 Memory-Mapped I/O and the Device Interface

The simplest possible way to let software talk to a device is to make the device's registers look exactly like memory locations. This idea is worth stating as a definition in its own right, because every technique in this chapter builds on it.

Definition (Memory-Mapped I/O). *In **memory-mapped I/O**, a device's data, status, and control registers occupy addresses in the CPU's normal address space. The CPU reads and writes them with ordinary load/store instructions — no special I/O instructions are needed (Hamacher, 6th ed., §3.1, p.96–97) [1].*

Every I/O device in this chapter connects to the CPU through a **device interface**: a small block of hardware supplying six functions in every case — a data register, a status register, a control register, address decoding (so the interface recognizes when the CPU addresses *it*), timing generation, and format conversion between the CPU's internal representation and whatever the device physically needs (Hamacher, 6th ed., §3.1.1/§7.4, p.97, p.238–239) [1]. Figure 8.1 shows this three-stage path: the CPU talks to the device interface over the address/data bus, and the interface in turn talks to the physical device using whatever signals that device needs. Concretely, the three registers the CPU actually sees are the **data register** (the byte actually transferred, e.g. `KBD_DATA` for the keyboard or `DISP_DATA` for the display), the **status register** (flags such as `KIN` or `DOUT`), and the **control register** (settings such as interrupt-enable bits).

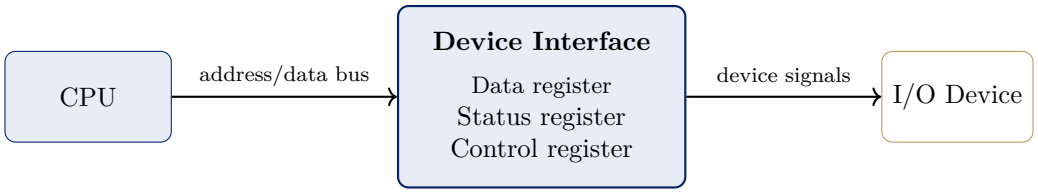

Figure 8.1: The device-interface model: a CPU reads/writes a device interface's data/status/control registers over the shared bus exactly as it would ordinary memory; the interface converts these transactions into whatever signals the physical device needs — cf. Hamacher, *Computer Organization and Embedded Systems*, 6th ed., Fig. 3.1–3.2, §3.1/§3.1.1, p.96–97 [1].

8.2.2 Isolated vs. Memory-Mapped I/O

Memory-mapped I/O is one of two standard ways of reaching a device's registers; the other is worth naming so the choice is explicit. In **isolated I/O**, devices live in a *separate* address space, reached only by dedicated I/O instructions rather than ordinary loads and stores — the classic IBM mainframe approach, and the standard alternative described in Mano Ch. 11 (chapter-level treatment; no page index for that chapter yet). Memory-mapped I/O, by contrast, needs no special instructions and uses the same load/store machinery the CPU already has; that simplicity is why the Hamacher examples in this chapter assume it. The trade-off between the two is summarized below.

	Memory-mapped I/O	Isolated I/O
Address space	Devices share the memory space	Separate I/O space
Instructions	Ordinary `Load`/`Store`	Dedicated I/O instructions
Simplicity	Simpler for the programmer	Extra hardware/instructions
Typical use	Most modern processors	Classic IBM mainframes

8.2.3 Program-Controlled I/O (Polling)

Given a device interface, the most direct thing software can do with a status register is watch it in a loop until it changes. This is program-controlled I/O.

Definition (Program-Controlled I/O (Polling)). *In **program-controlled I/O**, the CPU repeatedly reads a device's* status register *in a loop, checking a ready flag, until the device signals it is ready — then performs the data transfer itself (Hamacher, 6th ed., §3.1.2, p.97–100) [1].*

This chapter's running keyboard/display example introduces two status flags that recur through Sections 8.2 and 8.3: `KIN` is the keyboard-input status flag, set when a keystroke is ready in the register `KBD_DATA`; `DOUT` is the display-output status flag, set when the display is ready to accept the next character into `DISP_DATA`. In program-controlled I/O the CPU is the *only* thing that ever initiates a check — the device itself never signals the CPU proactively; it can only passively hold a flag until asked.

Example 8.2.1 (Polling Loops for `KIN` and `DOUT`). *Two polling loops, one for keyboard input and one for display output, both follow the identical three-instruction pattern (Hamacher, 6th ed., Fig. 3.3–3.5, §3.1.2–3.1.4, p.97–103) [1]:*

Keyboard input (`READWAIT` loop):

```
READWAIT: Test KIN
          Branch=0 READWAIT
          MoveByte KBD_DATA, R
```

Display output (`WRITEWAIT` loop):

```
WRITEWAIT: Test DOUT
           Branch=0 WRITEWAIT
           MoveByte R, DISP_DATA
```

Walking the keyboard loop instruction by instruction makes explicit what each pass actually does:

1. *`Test KIN` reads the keyboard's status register and sets the CPU's condition flags according to whether `KIN`$=1$ (a keystroke is waiting) or `KIN`$=0$ (nothing new).*
2. *`Branch=0 READWAIT` inspects those flags: if `KIN`$=0$, control jumps back to the `Test KIN` instruction and the whole two-instruction pair repeats; if `KIN`$=1$, execution falls through to the next line.*
3. *`MoveByte KBD_DATA, R` only executes once `KIN`$=1$ has finally been observed; it copies the waiting keystroke out of `KBD_DATA` into a CPU register `R`, completing the transfer.*

The display loop is the mirror image: it spins on `DOUT` instead of `KIN`, and once the display signals it is ready, `MoveByte R, DISP_DATA` pushes a character out *rather than pulling one* in. *In both loops, every pass before the flag finally flips re-executes the same `Test`/`Branch` pair and accomplishes nothing beyond re-reading a status flag that has not changed — the CPU is spinning, doing no other useful work while it waits [1].*

Example 8.2.2 (Quantifying Polling's Waste)**.** *A typist produces roughly 5 keystrokes per second, i.e. one keystroke every 0.2 seconds. A 2 GHz CPU executes* 2×10^9 *instructions per second, and each* `READWAIT` *iteration is 2 instructions (*`Test KIN` *and the failed* `Branch`*) before the successful pass adds the third. Between two keystrokes the CPU can execute*

$$0.2\ s \times 2 \times 10^9\ \textit{instr/s} = 4 \times 10^8\ \textit{instructions},$$

which at 2 instructions per failed iteration is on the order of 2×10^8 *(hundreds of millions) of* `READWAIT` *iterations. Every single one of them re-reads a status flag that has not changed since the last check, accomplishing* nothing *useful. Put another way, the fraction of the CPU consumed by polling this one keyboard is* $2 \times 10^8\ ns \div 2 \times 10^8\ ns = 1.0$ *— 100% of the CPU's time is spent waiting on this single device. Polling is simple — it requires no extra hardware beyond the status register itself — but it wastes CPU time in direct proportion to how much slower the device is than the CPU. It is a reasonable design when a single slow device has nothing else competing for the CPU's attention, and a poor one at any larger scale.*

8.3 Interrupt-Driven I/O

Polling wastes cycles because the CPU is the only party permitted to initiate anything. The obvious alternative is to let the device signal the CPU instead of the CPU repeatedly asking the device. That reversal of initiative is interrupt-driven I/O.

8.3.1 Definition and the Mechanics of a Transfer of Control

Definition (Interrupt). *An* ***interrupt*** *is a signal from a device that suspends the CPU's current program, transfers control to a fixed handler — the* ***interrupt-service routine (ISR)*** *— and resumes the original program afterward, exactly where it left off (Hamacher, 6th ed., §3.2, p.103–104) [1].*

Figure 8.2 shows this transfer of control as a time-flow diagram: the main program runs along its own timeline until the interrupt is accepted, at which point control jumps to the ISR's timeline; once the ISR finishes, a return-from-interrupt instruction (`RTI`) jumps back to the main program at the point immediately after it was suspended. Two pieces of new notation matter here. The gap between the moment a device requests an interrupt and the moment the ISR actually begins executing is the **interrupt latency**. The CPU registers the ISR is about to modify are first copied out into **shadow registers**, so the interrupted program's state survives the ISR untouched [1].

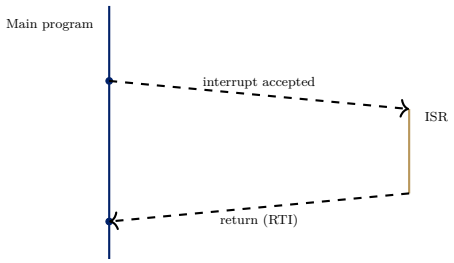

Figure 8.2: Interrupt as a transfer of control: the main program is suspended when the interrupt is accepted, control jumps to the ISR, and `RTI` returns control to the main program exactly where it left off — cf. Hamacher, *Computer Organization and Embedded Systems*, 6th ed., Fig. 3.6, §3.2, p.103–104 [1].

8.3.2 Enabling and Disabling Interrupts

An interrupt request from a device does not automatically suspend the CPU; two independent switches must both agree before it is honored (Hamacher, 6th ed., §3.2.1, p.106–107) [1]:

- `IE` — a single interrupt-enable bit inside the processor status register (`PS`). When `IE`= 0, the CPU ignores *every* interrupt request, regardless of which device sent it.
- A per-device interrupt-enable bit inside that specific device's own control register — `KIRQ` for the keyboard, `DIRQ` for the display. This bit controls whether *this particular* device is allowed to request an interrupt at all.

A request reaches the CPU only when both switches agree: the device's own enable bit *and* `PS.IE` must both be set. Plainly: `IE` is the master switch; each device also has its own little switch, and both must be on for the request to arrive. For example, `KIRQ`= 1 and `IE`= 1 means a keystroke will interrupt; `IE`= 0 means it waits. This two-level gating means a programmer can globally mask all interrupts with one bit (useful, for instance, while updating shared data structures that an ISR must not see half-updated) while separately deciding, device by device, which sources are even allowed to ask.

8.3.3 Worked Example: A Five-Step Interrupt-Handling Scenario

Example 8.3.1 (Handling One Keyboard Interrupt, Step by Step)**.** *Hamacher's 6th edition works through interrupt handling as a five-step scenario (§3.2.1, p.106–107) [1]. Applying it concretely to the keyboard, with `KIRQ`= 1 and `PS.IE`= 1 already set:*

1. ***Device signals readiness.*** *The keyboard sets its status flag `KIN`= 1 in its status register, and because `KIRQ`= 1, it also raises an interrupt-request line to the CPU.*
2. ***CPU finishes what it was doing.*** *The CPU does not react mid-instruction; it always completes whatever instruction is currently executing first, then checks `PS.IE`.*
3. ***CPU accepts the interrupt.*** *Since `PS.IE`= 1, the CPU saves `PC` (and any registers the ISR is about to clobber) into shadow registers, clears `PS.IE` to 0 (so a second interrupt cannot interrupt this one, unless the ISR explicitly re-enables it), and loads `PC` with the keyboard ISR's starting address.*
4. ***ISR services the device.*** *The keyboard ISR executes `MoveByte KBD_DATA, R` to read the pending keystroke, which as a side effect clears `KIN` back to 0, and then executes a return-from-interrupt instruction.*
5. ***Return-from-interrupt restores the original program.*** *`RTI` restores the saved `PC` from the shadow register and re-enables interrupts (`PS.IE`← 1). The interrupted program resumes at exactly the instruction it would have executed next, with no memory of having been interrupted.*

Compared to Example 8.2.1's `READWAIT` loop, the CPU here does zero work while waiting for a keystroke — it only spends time on the interrupt after *`KIN` actually becomes 1. The CPU never polls; it goes on computing until the device rings the bell.*

8.3.4 Handling Multiple Devices

A single interrupt-request line tells the CPU *that* something needs attention, but not *which* device raised it. Hamacher's 6th edition presents two designs for resolving this (§3.2.2, p.107–109) [1]:

- **Polling the sources.** On any interrupt, the ISR checks each device's status flag in turn, in some fixed order, until it finds the one that is set. This needs no extra hardware, but the fixed check order means dispatch time grows with the number of devices — the last device checked pays the largest latency.
- **Vectored interrupts.** Each device supplies its own identifying code as part of the interrupt request. Hardware uses that code to index directly into an **interrupt-vector table** — a table of ISR starting addresses, one entry per device — and jumps straight to the correct ISR.

Vectored interrupts trade a small amount of extra hardware (the vector table itself, plus the lookup logic) for constant-time dispatch, regardless of how many devices are attached — the CPU jumps directly to the right ISR instead of walking a checklist. Plainly: polling-the-sources asks everyone "was it you?" in order; a vector table skips the asking and goes straight to the right ISR.

8.3.5 Interrupt Nesting and Priority

When more than one device can interrupt, and interrupts can themselves be interrupted, the processor needs a systematic way to decide who goes first. Figure 8.3 shows the register-level mechanism: each device's IRQ line feeds a bit inside `IPENDING`, a per-device interrupt-enable mask lives in `IENABLE`, and both converge on a priority encoder that, together with the vector-table lookup, produces the winning device's ISR starting address (Hamacher, 6th ed., §3.2.4, Fig. 3.7, p.110–111) [1]. This is the direct register-level analogue of Week 7's "two sources feed `MPC`" diagram: multiple candidate signals converge on one decision point.

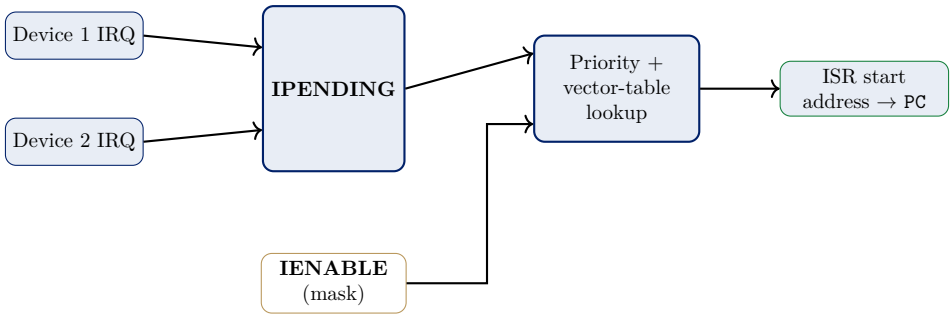

Figure 8.3: Interrupt dispatch with priority: device IRQ lines set bits in `IPENDING`; `IENABLE` masks which pending requests the priority encoder considers; the encoder plus vector-table lookup produces the winning ISR's starting address — cf. Hamacher, *Computer Organization and Embedded Systems*, 6th ed., Fig. 3.7, §3.2.4, p.110–111 [1].

What the diagram shows: `IPENDING` latches *which* devices currently want an interrupt; `IENABLE` decides *which of those* are allowed through; the priority encoder picks the highest-priority allowed one; the vector table gives its ISR address. When two devices request simultaneously, the priority encoder grants the higher-priority one first; `IPENDING` keeps the other device's request latched — not lost — until that device is eventually serviced.

Example 8.3.2 (Simultaneous Requests and Nesting)**.** *Device 1 has higher priority than Device 2. Both request an interrupt on the same cycle, and Device 1's ISR is still running when Device 2's request would otherwise be granted. Two cases follow directly from whether* nesting *is enabled:*

- ***Nesting allowed, and Device 2 outranks Device 1's ISR's running priority level.*** *Device 2 can interrupt Device 1's ISR itself — the identical five-step scenario of Example 8.3.1 applies recursively, with Device 1's ISR now playing the role of the "main program" being suspended.*
- ***Nesting disabled, or Device 2 is lower priority than Device 1's currently-running ISR.*** *Device 2's request stays latched in* `IPENDING`*. It is not lost or re-requested; it simply waits until Device 1's ISR finishes and re-enables interrupts (*`PS.IE`$\leftarrow 1$ *via* `RTI`*), at which point the priority encoder re-evaluates* `IPENDING` *and grants Device 2.*

8.3.6 Exceptions: The General Form of an Interrupt

The interrupt mechanism is not special to I/O devices; it is the same machinery the CPU uses for any event that must suspend the current program. Naming this generalization matters because it lets one mechanism serve many different sources.

Definition (Exception). *An* ***exception*** *is any event that interrupts the normal flow of control — an I/O interrupt, but also divide-by-zero, an illegal instruction, a page fault, or a timer tick. The interrupt mechanism is the same; only the* source *differs (Hamacher, 6th ed., §3.2.6, p.116–119; P&H, ARM ed., Ch. 4, chapter-level) [1, 2].*

Plainly, an interrupt is just one kind of exception — the "a device needs me" kind. The CPU handles all of them with the same jump-to-handler machinery from Figure 8.2: save state, service, restore. This is also the bridge later lectures will reuse — exceptions are how the operating system gets control back from user programs.

8.3.7 Polling vs. Interrupts: Trade-offs

	Polling	**Interrupts**
Who initiates	CPU (asks repeatedly)	Device (signals once, when ready)
CPU cost while waiting	Wasted spin cycles	Zero — CPU runs other code
Extra hardware	None	`IE`, per-device enables, vector table
Response latency	Depends on poll rate	Interrupt latency (usually smaller)
Best fit	One slow device, nothing else to do	Multiple/unpredictable devices

Both techniques solve the same problem — *someone* has to notice when a device is ready — but move the initiative in opposite directions. Interrupts do not eliminate the waiting cost; they relocate it from wasted CPU cycles to the hardware that detects readiness and raises the request line.

8.4 The Hardware View: Synchronous and Asynchronous Bus Protocols

Sections 8.2–8.3 took the device interface for granted and asked only *when* the CPU checks or is notified. This section drops one level, to the wires themselves: when a device interface says "I'm not ready yet" or "here is the data," how is that actually communicated on the bus?

8.4.1 Bus Structure: Master and Slave Roles

Every I/O interface in Figure 8.1 in fact hangs off a single shared **bus** — a set of address, data, and control lines that every attached device listens to (Hamacher, 6th ed., §7.1, Fig. 7.1–7.2, p.228) [1]. A **bus protocol** defines how a **master** (the device driving the transfer — usually the CPU) and a **slave** (the device responding to it) coordinate a single data transfer without colliding. This section's question is: when a master says "take this data" or "give me that data," how does the slave say "I'm not ready yet" or "here it is"? Two different answers exist: **synchronous** and **asynchronous** bus protocols (Hamacher, 6th ed., §7.2, p.229) [1].

8.4.2 Synchronous Bus

Definition (Synchronous Bus). *On a **synchronous bus**, all devices share a common **bus clock**. The master places an address/data value on the bus at a fixed clock edge; every slave samples the bus at that* same *edge, with no per-transfer negotiation (Hamacher, 6th ed., §7.2.1, p.230–233) [1].*

Figure 8.4 shows the timing diagram for a synchronous transfer that must wait on a slow slave. Three signals run in parallel over time: the shared clock, the address/data lines (held valid across the whole transfer), and a `Slave-ready` line that stays low — meaning "not yet" — for as many extra clock cycles as the slow device needs, then rises once the data is actually valid.

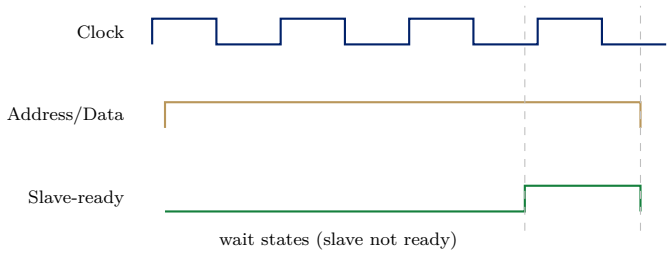

Figure 8.4: Synchronous-bus timing with wait states: the address/data lines are held valid across the whole transfer while `Slave-ready` stays low for several extra clock periods until the slow slave is finally ready — cf. Hamacher, *Computer Organization and Embedded Systems*, 6th ed., Fig. 7.3–7.5, §7.2.1, p.230–233 [1].

Example 8.4.1 (Reading the Synchronous-Bus Wait-State Timeline)**.** *Following Figure 8.4 left to right against the running 4096-byte block-transfer example: at the first clock edge (time 0), the master places the target address on the address/data lines and they remain valid there for the rest of the diagram.* `Slave-ready` *starts low, meaning the slave has not yet placed (or accepted) valid data. For several full clock periods — the "wait states" region — the clock continues to toggle every half-period, but* `Slave-ready` *stays low each time, because the slow device physically has not finished. Only at the clock edge marked by the first dashed vertical line does* `Slave-ready` *finally rise; the master samples the bus at the* next *clock edge (the second dashed line) and the transfer of that one word completes. A fast device would need zero wait states —* `Slave-ready` *would already be high at the very first edge — while a slower device simply holds* `Slave-ready` *low for as many extra clock cycles as it needs; the master's logic does not otherwise change.*

8.4.3 Asynchronous Bus (Full Handshake)

Definition (Asynchronous Bus (Full Handshake)). *An **asynchronous bus** has no shared clock. Master and slave exchange explicit **Master-ready/Slave-ready** signals, each one changing only* in response to *the other — a **full handshake** (Hamacher, 6th ed., §7.2.2, p.233–236) [1].*

Figure 8.5 shows this exchange as a four-step interlocked ladder between a Master column and a Slave column, where each arrow only fires after the previous one has been observed.

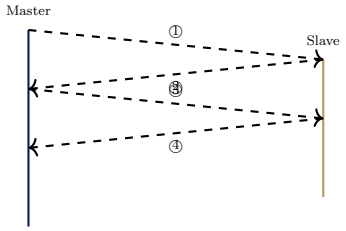

Figure 8.5: The asynchronous full handshake as a four-step interlocked exchange between master and slave — cf. Hamacher, *Computer Organization and Embedded Systems*, 6th ed., Fig. 7.6–7.7, §7.2.2, p.233–236 [1].

① Master-ready asserted
② Slave-ready (data valid)
③ Master-ready withdrawn (latched)
④ Slave-ready withdrawn

Example 8.4.2 (Walking the Four-Step Handshake)**.** *Applying the four steps of Figure 8.5 to one word of the running 4096-byte block transfer, in strict sequence, with each step waiting for the previous one to be observed before it can fire:*

1. ***Master-ready asserted.*** *The master places the address/data on the bus and asserts* `Master-ready`*, announcing "I have placed a value; take it (or: I am ready to receive)."*

2. ***Slave-ready (data valid).*** *Only after observing* `Master-ready`*, the slave completes its side of the transfer (latching the data, or supplying it) and asserts* `Slave-ready`*, announcing "done, data is valid."*

3. ***Master-ready withdrawn (latched).*** *Only after observing* `Slave-ready`*, the master withdraws* `Master-ready` *— it has now recorded that the transfer completed.*

4. ***Slave-ready withdrawn.*** *Only after observing* `Master-ready` *drop, the slave withdraws* `Slave-ready`*, returning both lines to their idle state and readying the bus for the next word.*

*Because each step is triggered only by observing the previous one — never by a shared clock edge — **bus skew** (different wires physically arriving at slightly different times) never causes a false sample: a device simply waits until it actually observes the other side's signal, however long that takes. This is the crucial difference from Figure 8.4's synchronous protocol, where every device must sample at the same fixed clock edge regardless of individual wire delays.*

8.4.4 Synchronous vs. Asynchronous: Trade-offs

	Synchronous	**Asynchronous**
Timing reference	Shared bus clock	Explicit handshake, no clock
Slow-device handling	Extra wait-state clock cycles	Handshake simply takes longer
Skew sensitivity	Sensitive — must beat clock period	Immune — self-timed
Per-transfer overhead	Low (one clock edge)	Higher (two round trips)
Best fit	Devices of similar, known speed	Mixed-speed / unpredictable devices

Both protocols solve the identical coordination problem — "how does the slave say it's ready?" — but with opposite mechanisms: a shared clock that every device trusts implicitly, versus an explicit conversation that assumes nothing about timing at all.

Example 8.4.3 (Why Not Always Use Asynchronous?)**.** *Asynchronous handshaking tolerates skew and adapts automatically to any device speed, which sounds strictly better — so why do most high-speed processor buses use synchronous protocols instead? The answer is in the "per-transfer overhead" row of the trade-off table above: every asynchronous transfer pays* two full round-trip delays *— steps 1–2 of Example 8.4.2 form one round trip, and steps 3–4 form a second. For a device that is consistently fast, both round trips are pure overhead that a synchronous bus never pays; the synchronous bus only loses ground when a device is unusually slow (extra wait states, as in Example 8.4.1) or when the clock period must shrink to accommodate skew across many devices. High-speed buses connecting devices of known, similar speed — exactly the synchronous bus's "best fit" row — therefore favor the cheaper synchronous protocol.*

8.5 Direct Memory Access and I/O Processors

8.5.1 The Cost Interrupts Still Pay

Interrupts (Section 8.3) removed the CPU's *waiting* cost, but not its *involvement* in actually moving data. This is easiest to see by returning to the running 4096-byte block-transfer example and asking what interrupt-driven I/O costs when the transfer is one word at a time, four bytes per word.

Example 8.5.1 (Counting ISR Invocations for a Block Transfer)**.** *With interrupt-driven I/O, the ISR from Example 8.3.1 executes once per word: read status, read/write data, update a memory pointer, decrement a word counter, and return. A 4096-byte disk transfer, moved four bytes at a time, therefore requires*

$$4096 \div 4 = 1024$$

separate ISR invocations for a single *block transfer. Interrupts eliminated the CPU spinning idly between words, but the CPU is still fully engaged for all 1024 of those short ISR executions — one context switch into the ISR and back out, per word. The next technique removes the CPU from the data-movement path itself.*

8.5.2 Bus Arbitration: A Third Role for a Device

DMA depends on a capability not yet introduced: an I/O device, not just the CPU, becoming bus master. **Bus arbitration** is the mechanism that decides which of several candidate masters gets the bus on any given cycle (Hamacher, 6th ed., §7.3, Fig. 7.8–7.9, p.237–238) [1]. Figure 8.6 shows three candidate masters, each able to assert a bus-request line (`BR`), feeding into an arbiter; the arbiter grants the bus to exactly one requester at a time, by priority, over a bus-grant line (`BG`).

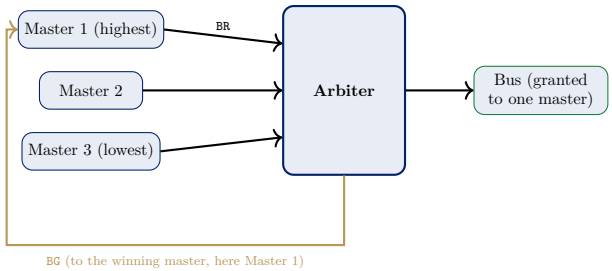

Figure 8.6: Bus arbitration: three candidate masters assert BR; the arbiter grants BG to exactly one requester, by priority, at a time — cf. Hamacher, *Computer Organization and Embedded Systems*, 6th ed., Fig. 7.8–7.9, §7.3, p.237–238 [1].

Example 8.5.2 (Three-Master Priority Arbitration)**.** *Suppose Masters 1, 2, and 3 (priority highest to lowest, as in Figure 8.6) all assert BR on the same cycle, competing for the bus. The arbiter examines all three requests simultaneously and grants BG to Master 1, the highest-priority requester — Masters 2 and 3 continue holding BR asserted, waiting. Once Master 1 finishes its transfer and releases the bus, the arbiter re-evaluates the still-pending requests from Masters 2 and 3 and grants BG to Master 2 next, by the same priority rule; Master 3 is served last, once both higher-priority masters have released the bus. Any device wired into the BR/BG scheme can become bus master this way — not only the CPU — which is exactly the capability DMA needs. Being bus master is how a device talks to memory* directly*.*

8.5.3 From Arbitration to Direct Memory Access

Once a device can become bus master via arbitration, it can transfer data *directly to or from memory*, with no CPU instruction ever touching the data. The CPU is interrupted only *once*, when the entire block finishes, not once per word as in Example 8.5.1. This edition of Hamacher's text describes exactly this mechanism in §7.3 (Arbitration) but never names it "DMA" — the term and its standard treatment below follow Stallings Ch. 8, per the syllabus and the project's textbook-grounding rule, since this book's index does not cover DMA/IOP by name [1].

Definition (Direct Memory Access (DMA) — general/standard treatment, Stallings Ch. 8). *A **DMA controller** is given a source/destination address and a word count by the CPU, then becomes bus master (via the arbitration mechanism of Figure 8.6) and transfers the entire block directly to/from memory, interrupting the CPU only when the* block *— not each word — is complete [3].*

Plainly: the CPU says "copy this block," then goes back to work; the DMA controller does the copying and rings the bell when done.

8.5.4 The DMA Controller: What's Inside

To do its job, the DMA controller carries four registers, all written once by the CPU at setup and owned by the controller for the rest of the transfer. Figure 8.7 shows where they live: the DMA controller sits between the CPU and the I/O device on the bus, and can become bus master to reach memory directly.

- **Source / destination address** — where the data comes from and goes to (memory address and device, or vice versa).

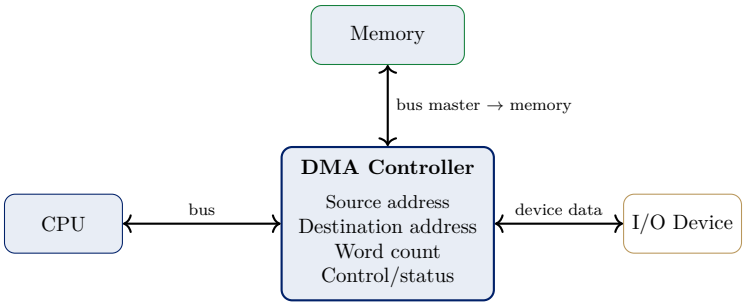

Figure 8.7: The DMA controller and its four registers: source/destination address, word count, and control/status. The CPU writes them once at setup; the controller becomes bus master to move data directly between the device and memory — general/standard treatment, Stallings Ch. 8 [3].

- **Word count** — how many words remain to transfer; decremented after every word.
- **Control/status** — transfer direction, mode, and the "block done" flag that interrupts the CPU.

8.5.5 How a DMA Transfer Runs: Step by Step

Example 8.5.3 (The Six-Step DMA Transfer Sequence)**.** *Copying the running 4096-byte block from disk to memory, the CPU appears exactly twice — at setup and at completion — and never touches the data itself [3]:*

1. ***Setup.*** *The CPU writes the DMA controller's registers: source address (disk), destination address (memory), word count (1024), and direction (read).*
2. ***Request.*** *The DMA controller asks for the bus by asserting* `BR` *(bus request).*
3. ***Grant.*** *The arbiter answers with* `BG` *(bus grant); the DMA controller becomes bus master (Hamacher, 6th ed., §7.3, p.237–238) [1].*
4. ***Transfer one word.*** *The DMA controller reads one word from the device and writes it to memory (or the reverse), using one or two bus cycles.*
5. ***Decrement.*** *It subtracts 1 from the word count and, if the count is still* > 0*, returns to step 2 for the next word.*
6. ***Done.*** *When the count reaches 0, the DMA controller drops* `BR` *and raises a* completion interrupt *— the CPU's only involvement after setup.*

Compare with polling and interrupts: the CPU now appears exactly twice, at the start and the end.

8.5.6 DMA Transfer Modes

The DMA controller can choose how aggressively it holds the bus. The trade-off is always the same — speed of the transfer versus how much it starves the CPU — and three standard modes span the range (general/standard treatment, Stallings Ch. 8) [3]:

Burst (block) mode. The DMA controller holds the bus for the *entire* block — all 1024 words in one continuous burst, then releases the bus and interrupts the CPU. Fastest possible block transfer, but the CPU cannot reach memory at all for the duration of the burst.

Cycle-stealing (single) mode. The DMA controller grabs the bus for *one word*, releases it, and re-requests — so the CPU can use the bus between DMA words. Slower per block, but the CPU keeps running between transfers.

Transparent (hidden) mode. The DMA controller transfers *only during CPU cycles that do not use the bus at all* — the CPU never even notices the DMA happening. Slowest transfer rate, but zero effect on the CPU.

Plainly: burst = "whole job now, don't bother me"; cycle stealing = "one word, then back off"; transparent = "only when you're not looking."

	Burst	**Cycle stealing**	**Transparent**
Bus held	Whole block	One word	Only idle CPU cycles
Transfer speed	Fastest	Medium	Slowest
Effect on CPU	Starved for the block	Brief pauses per word	None

8.5.7 DMA's Cost Reduction on the Running Block Transfer

Example 8.5.4 (DMA's Cost Reduction on the Running Block Transfer)**.** *Revisiting Example 8.5.1's 4096-byte transfer under DMA: the CPU's only involvement is a single setup step (write the source/destination address and the word count of 1024 into the DMA controller's registers) and a single completion interrupt once the DMA controller has moved all 1024 words. This is a reduction from 1024 ISR invocations to 2 CPU events total (1 setup + 1 completion interrupt) — a constant cost regardless of block size, in contrast to interrupt-driven I/O's cost, which grows linearly with the number of words transferred.*

Putting instruction counts on it makes the saving concrete: if one ISR takes 200 CPU instructions, interrupt-driven I/O spends $1024 \times 200 = 204{,}800$ *instructions moving the block, while DMA spends one setup plus one completion ISR — roughly 400 instructions — about* $500\times$ less *CPU work. The CPU that used to spend its whole time servicing the disk can now run the user's program instead.*

The trade-off is that the DMA controller and the CPU now compete for the same memory bus while the transfer is in progress (the bus is occupied for the same 1024 transfer cycles either way), and a physical DMA controller is extra hardware neither polling nor plain interrupts require.

8.5.8 I/O Processors (IOP)

Definition (I/O Processor (IOP) — general/standard treatment, Stallings Ch. 8). *An* ***I/O processor*** *generalizes DMA one step further: a dedicated processor, given a whole* channel program *(a short program describing several transfers, data formats, and simple decisions), executes it independently — interrupting the main CPU only when the* entire program *completes, not per block [3].*

Plainly: DMA copies one block; an IOP runs a whole to-do list of I/O jobs and reports back when the list is done. Where DMA offloads only the *moving* of data, an IOP offloads *managing a sequence* of I/O operations. The distinction is how much decision-making is delegated away from the main CPU, not merely how the data physically moves: a DMA controller executes one preconfigured block transfer and stops; an IOP can execute a whole program of transfers, formats, and conditional decisions on its own, reporting back to the CPU only when that entire program is done.

8.5.9 Four I/O Techniques, Head to Head

	Polling	Interrupts	DMA	IOP
CPU cost / word	Every poll	Every word (ISR)	None (setup only)	None
CPU cost / block	—	Thousands of ISRs	One interrupt	One interrupt
Extra hardware	None	Vector table, IE	DMA controller	Dedicated processor
Best fit	One slow device	Multiple/sporadic devices	Bulk block transfer	Complex I/O subsystems

All four techniques answer the same core question — who watches the device, and how much of the CPU does watching cost? — with a monotonically decreasing amount of CPU involvement, purchased with a monotonically increasing amount of dedicated hardware.

Example 8.5.5 (Choosing a Technique for Two Different Devices)**.** *A system reads one temperature-sensor value per second (a low rate, and the application can tolerate some latency in reading it) and, separately, copies a 10 MB file from disk to memory (a high rate, throughput-critical operation). Applying the head-to-head comparison above to each:*

- ***Temperature sensor: interrupts.*** *At one event per second, the ISR overhead per event (a handful of instructions, per Example 8.3.1) is utterly negligible relative to the CPU's billions of instructions per second, and the CPU remains completely free to do other work the rest of the time. Polling would waste cycles for no benefit (as in Example 8.2.2); DMA/IOP hardware would be pure overkill for a single value per second.*
- ***Disk copy: DMA.*** *At 10 MB moved in 4-byte words, interrupt-driven I/O would need on the order of* 2.5×10^6 *ISR invocations (following the arithmetic of Example 8.5.1 scaled up), which would dominate CPU time entirely. DMA reduces the CPU's involvement to one setup and one completion interrupt regardless of the block's size, exactly the trade-off Example 8.5.4 quantified.*

The rule of thumb: match the technique to the rate *and* size *of the transfer, not to habit.*

8.6 Synthesis: The Four Threads Meet

Every design decision in this chapter moves the cost of "someone has to notice the device is ready, and someone has to move the data" to a different place — CPU cycles, dedicated hardware, or per-transfer wire delay — never eliminating it outright.

Polling vs. interrupts (the programmer's view, Sections 8.2–8.3): polling wastes CPU cycles asking a status flag that has not changed; interrupts let the device ask instead, at the cost of the vector-table and priority hardware from Figure 8.3.

Synchronous vs. asynchronous (the bus hardware, Section 8.4): a shared clock is cheap per transfer but sensitive to skew and slow devices (Figure 8.4); an explicit handshake is skew-immune but pays two full round trips on every single transfer (Figure 8.5).

Interrupts vs. DMA/IOP (who actually moves the data, Section 8.5): interrupts still touch every single word (Example 8.5.1); DMA and IOP remove the CPU from the data path entirely, trading extra hardware (a DMA controller, or a whole dedicated processor) for near-zero per-word CPU cost (Example 8.5.4).

None of these three trade-offs is free — exactly the pattern Weeks 5–7 already established for bus organization and control-unit design, now applied to the world outside the chip rather than inside it.

Weeks 1–8 together built a CPU that computes, controls itself, and now moves data to and from slow external devices. Week 9 asks the next question in the same speed-mismatch pattern: once

data is in memory, how do we make *memory itself* fast enough to keep up with the CPU? DRAM is far slower than the CPU, reproducing the identical speed-mismatch problem this chapter solved for I/O devices, but for memory instead — cache memory, mapping functions, and replacement policies are Week 9's answer.

8.6.1 Summary

Memory-mapped I/O places a device's data/status/control registers directly in the normal address space (the **isolated-I/O** alternative needs separate I/O instructions and a separate address space); **program-controlled I/O (polling)** repeatedly tests a status flag (`KIN`, `DOUT`) until it changes. **Interrupts** let the device signal readiness instead: `PS.IE` and per-device enables (`KIRQ`/`DIRQ`) gate delivery, and **vectored interrupts** use `IPENDING`/`IENABLE` plus an interrupt-vector table for constant-time dispatch regardless of device count; an **exception** generalizes the same mechanism to non-I/O events. On the hardware side, a **synchronous bus** uses a shared clock and wait states for slow slaves, while an **asynchronous bus** uses a `Master-ready`/`Slave-ready` full handshake — skew-immune, but at a higher per-transfer overhead of two round trips. **Bus arbitration** (`BR`/`BG`) lets any device become bus master, which is the functional basis for **DMA** — a controller with source/destination address, word count, and control/status registers, running a six-step transfer in burst, cycle-stealing, or transparent mode, one setup plus one block-completion interrupt — and the **IOP** (a whole channel program executed independently, general/standard treatment per Stallings Ch. 8). All four techniques answer the same underlying question — how much of the CPU does watching and moving the data cost, and how much hardware buys that cost down?

Exercises

1. A printer produces output slowly enough that the CPU can service it with either polling or interrupts. Explain, using the reasoning of Example 8.2.2, why interrupts become strictly preferable as the CPU's clock rate increases relative to the printer's fixed speed, even though the printer's own speed never changes.

2. Extend Example 8.3.1's five-step scenario to a *second* interrupt-capable device, a mouse, with lower priority than the keyboard and `PS.IE` initially = 1. Write out, step by step, what happens if the mouse raises an interrupt request while the keyboard's ISR is already running and interrupt nesting is disabled.

3. A synchronous bus transfer (as in Figure 8.4) needs 3 wait-state clock cycles at a 200 MHz bus clock before `Slave-ready` rises. Compute the total time, in nanoseconds, from the first clock edge until the transfer completes (the edge at which the master samples valid data), counting the wait states plus the one clock period needed to actually sample.

4. Following Example 8.5.2, suppose a fourth master (lowest priority, below Master 3) also asserts `BR` on the same cycle as Masters 1–3. In what order are all four masters granted the bus, assuming each releases the bus promptly after its own transfer and no new requests arrive in the meantime?

5. Using Example 8.5.4's method, compute the number of ISR invocations interrupt-driven I/O would need, and the number of CPU events DMA would need, to transfer an 8192-byte block, again 4 bytes per word. Compare the two totals and state, in one sentence, which technique's cost is scale-independent and which is not.

6. A DMA controller is configured to copy a 2048-word block in cycle-stealing mode, one word per bus grant, on a bus where each one-word grant lasts 2 ns. In the time the DMA controller takes to move the whole block, how many total nanoseconds of bus time does it occupy? Would burst mode occupy more, less, or the same total bus time — and what is the difference in the two modes' effect on the CPU?

Solutions

1. As the CPU's clock rate rises, the number of instructions the CPU can execute during one printer inter-character interval grows proportionally (Example 8.2.2 showed $\sim 2 \times 10^8$ wasted iterations already at 2 GHz for a 5 Hz keyboard). A faster CPU polling the same fixed-speed printer wastes proportionally *more* absolute CPU cycles per printed character, even though the printer's own speed and the fraction of "useful" polls stays the same. Interrupts' cost, in contrast, is fixed at one ISR invocation per printer-ready event regardless of CPU clock rate, so interrupts' relative advantage over polling grows as CPU speed grows.
2. Step 1: mouse sets its status flag and, since its own enable bit is set, raises an interrupt-request line; `IPENDING` latches the mouse's request bit. Step 2: the CPU is currently inside the keyboard's ISR, which set `PS.IE`= 0 upon entry (per Example 8.3.1, step 3) — so the CPU is not currently checking `PS.IE` for new requests at all; even if it were, `IE`= 0 would mask every request. Step 3: because nesting is disabled (and even if it weren't, the mouse is lower priority than the keyboard), the mouse's request simply stays latched in `IPENDING`, unserviced. Step 4: the keyboard's ISR finishes and executes `RTI`, which restores `PC` and sets `PS.IE`← 1 again. Step 5: with interrupts now re-enabled, the priority encoder (Figure 8.3) re-evaluates `IPENDING`, finds the mouse's still-latched request, and only now grants it — the mouse's five-step scenario begins at this point, using the main program (not the keyboard's ISR) as the program being suspended.
3. One clock period at 200 MHz is $1/(200 \times 10^6) = 5$ ns. Three wait-state cycles cost $3 \times 5 = 15$ ns, and the final sampling edge occurs one more clock period after `Slave-ready` rises, adding another 5 ns. Total: $15 + 5 = 20$ ns from the first edge to the completed transfer (equivalently, 4 clock periods total).
4. Priority is Master 1 > Master 2 > Master 3 > Master 4 (lowest). Following Example 8.5.2's reasoning directly: the arbiter grants the highest-priority *still-pending* request each time a master releases the bus, so the order is Master 1, then Master 2, then Master 3, then Master 4 — strict priority order, since all four requests were pending simultaneously from the start and no new requests arrive to reorder the queue.
5. Interrupt-driven I/O: $8192 \div 4 = 2048$ ISR invocations — exactly double Example 8.5.1's count for the 4096-byte block, since the ISR count scales linearly with block size. DMA: still 2 CPU events total (1 setup + 1 completion interrupt) — identical to the 4096-byte case, because the DMA controller's cost to the CPU does not depend on how large the word count in its setup registers is. DMA's cost is scale-independent (constant in block size); interrupt-driven I/O's cost is not (linear in block size) — this is exactly the gap Example 8.5.4 identified, now confirmed to hold at a different block size too.
6. The DMA controller occupies 2048×2 ns = 4096 ns of bus time total, regardless of mode — burst and cycle stealing both move 2048 words at one 2 ns grant per word. The difference is *when* that bus time is taken: burst takes all 4096 ns in one continuous stretch (starving the CPU for the whole transfer), while cycle stealing interleaves the same 4096 ns of bus time with CPU bus accesses one word at a time, so the CPU keeps running between grants. Total bus occupancy is identical; the modes differ only in whether that occupancy is continuous or interleaved.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, Safwat G. Zaky, and Naraig Manjikian. *Computer Organization and Embedded Systems.* McGraw-Hill, 6th edition, 2012.

ISBN 978-0-07-338065-0. Bib key retained as Hamacher2002_computer_organization for citation-key stability across existing lectures; the file indexed under master_supporting_docs/CS401/supporting_books/Hamacher2002/ and page-cited by Week 8 is this 6th edition, not the 5th ed. (2002) the key name suggests – see that folder's index.md Edition notice, 2026-08-11.

[2] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.

[3] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.